Week 10: APIs, JSON, and Security

Overview

This week introduces you to working with external APIs, processing JSON data, and implementing security best practices. You'll learn how to make HTTP requests, parse API responses, and secure your applications against common threats.

Learning Objectives

By the end of this week, you should be able to:

- Make HTTP requests using Java's HttpClient
- Parse and create JSON data structures
- Validate and sanitize user input
- Implement rate limiting for API protection
- Apply authentication mechanisms
- Handle errors securely without information leakage

Why APIs and Security Matter

Real-world Applications:

- Mobile apps consuming backend APIs
- Microservices communicating with each other
- Third-party integrations (payment, weather, social media)
- Data exchange between systems

Security Importance:

- Protect user data from unauthorized access
- Prevent system abuse and resource exhaustion
- Maintain system integrity and availability
- Comply with security regulations and standards

Benefits:

- **Interoperability**: Systems can communicate regardless of technology
- **Scalability**: Independent scaling of API services
- **Flexibility**: Easy integration with third-party services
- **Security**: Controlled access to resources

Example Index

Getting Started

Start Here: Run `Week10ExampleIndex.java` to see all available examples.

```
mvn exec:java -Dexec.mainClass="com.example.basic.week10ApisSecurity.Week10ExampleIndex"
```

Example 1: HTTP Client

File: `HttpClientDemo.java`

What It Demonstrates:

- Using Java's built-in HttpClient (Java 11+)
- Making GET and POST requests
- Setting headers and timeouts
- Handling HTTP status codes
- Network error handling

Key Code Patterns:

```
// Create reusable HTTP client
HttpClient client = HttpClient.newBuilder()
        .connectTimeout(Duration.ofSeconds(10))
        .build();

// Build GET request
HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create("https://api.example.com/users/1"))
        .header("Accept", "application/json")
        .timeout(Duration.ofSeconds(30))
        .GET()
        .build();

// Send request and get response
HttpResponse<String> response = client.send(request,
        HttpResponse.BodyHandlers.ofString());

// Check status and handle response
if (response.statusCode() == 200) {
    String body = response.body();
    // Process successful response
} else if (response.statusCode() == 404) {
    // Handle not found
} else {
    // Handle other errors
}
```

POST Request Pattern:

```
String jsonBody = """
    {
        "name": "Alice",
        "email": "alice@example.com"
    }
    """;

HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create("https://api.example.com/users"))
        .header("Content-Type", "application/json")
        .POST(HttpRequest.BodyPublishers.ofString(jsonBody))
        .build();
```

Programming Principles:

- **Resource Management**: HttpClient is reusable and thread-safe
- **Builder Pattern**: Flexible request construction
- **Immutability**: HttpRequest objects cannot be modified after creation
- **Error Handling**: Comprehensive exception handling for network operations

How to Run:

```
mvn exec:java -Dexec.mainClass="com.example.basic.week10ApisSecurity.HttpClientDemo"
```

Example 2: JSON Processing

File: `JsonProcessingDemo.java`

What It Demonstrates:

- Creating JSON objects and arrays
- Parsing JSON strings
- Working with nested JSON structures
- Type-safe data extraction
- Handling JSON parsing errors

Key Code Patterns:

```java
import org.json.JSONObject;
import org.json.JSONArray;

// Creating JSON
JSONObject person = new JSONObject();
person.put("name", "Alice");
person.put("age", 28);
person.put("email", "alice@example.com");

// Adding nested object
JSONObject address = new JSONObject();
address.put("city", "New York");
address.put("zipCode", "10001");
person.put("address", address);

// Adding array
JSONArray skills = new JSONArray();
skills.put("Java");
skills.put("Python");
skills.put("SQL");
person.put("skills", skills);

// Pretty print JSON
System.out.println(person.toString(2));  // 2-space indentation
```

Parsing JSON Pattern:

```
String jsonString = """
    {
        "name": "Bob",
        "age": 30,
        "address": {
            "city": "Boston",
            "state": "MA"
        },
        "skills": ["Java", "Python", "SQL"]
    }
    """;

try {
    JSONObject obj = new JSONObject(jsonString);

    // Extract simple values
    String name = obj.getString("name");
    int age = obj.getInt("age");

    // Extract nested object
    JSONObject address = obj.getJSONObject("address");
    String city = address.getString("city");

    // Extract array
    JSONArray skills = obj.getJSONArray("skills");
    for (int i = 0; i < skills.length(); i++) {
        String skill = skills.getString(i);
        System.out.println(skill);
    }

} catch (JSONException e) {
    System.err.println("JSON parsing error: " + e.getMessage());
}
```

Safe Field Access:

```
// Check if field exists
if (obj.has("email")) {
    String email = obj.getString("email");
}

// Use opt() methods with defaults
String email = obj.optString("email", "no-email@example.com");
int age = obj.optInt("age", 0);
boolean isActive = obj.optBoolean("isActive", false);
```

Programming Principles:

- **Type Safety**: Proper type checking and conversion
- **Error Handling**: Always catch JSONException
- **Validation**: Check for required fields before accessing
- **Readability**: Use pretty printing for debugging

How to Run:

```
mvn exec:java -Dexec.mainClass="com.example.basic.week10ApisSecurity.JsonProcessingDemo"
```

Example 3: API Security

File: `ApiSecurityDemo.java`

What It Demonstrates:

- Input validation and sanitization
- Rate limiting with token bucket algorithm
- API Key authentication
- Basic authentication (Base64)
- Secure error handling

Key Code Patterns:

Input Validation:

```
public class InputValidator {
    private static final Pattern EMAIL_PATTERN =
        Pattern.compile("^[A-Za-z0-9+_.-]+@([A-Za-z0-9.-]+\\.[A-Za-z]{2,})$");

    public boolean isValidEmail(String email) {
        if (email == null || email.trim().isEmpty()) {
            return false;
        }
        return EMAIL_PATTERN.matcher(email).matches();
    }

    public String sanitizeInput(String input) {
        if (input == null) {
            return "";
        }
        // Remove potentially dangerous characters
        return input.replaceAll("[<>\"';]", "").trim();
    }

    public void validateUserInput(String name, int age, String email)
            throws ValidationException {
        if (name == null || name.trim().isEmpty()) {
            throw new ValidationException("Name cannot be empty");
        }
        if (age < 0 || age > 150) {
            throw new ValidationException("Age must be between 0 and 150");
        }
        if (!isValidEmail(email)) {
            throw new ValidationException("Invalid email format");
        }
    }
}
```

Rate Limiting:

```java
public class RateLimiter {
    private final Map<String, TokenBucket> buckets = new ConcurrentHashMap<>();
    private final int maxRequests;
    private final long windowMs;

    public RateLimiter(int maxRequests, long windowMs) {
        this.maxRequests = maxRequests;
        this.windowMs = windowMs;
    }

    public boolean isAllowed(String clientId) {
        TokenBucket bucket = buckets.computeIfAbsent(clientId,
            k -> new TokenBucket(maxRequests, windowMs));
        return bucket.tryConsume();
    }
}

// Usage
RateLimiter limiter = new RateLimiter(100, 60000);  // 100 requests per minute
if (limiter.isAllowed("user123")) {
    // Process request
} else {
    // Return 429 Too Many Requests
}
```

API Key Authentication:

```
public class ApiKeyAuthenticator {
    private final Map<String, String> apiKeys = new ConcurrentHashMap<>();

    public void registerApiKey(String userId, String apiKey) {
        apiKeys.put(apiKey, userId);
    }

    public boolean authenticate(String apiKey) {
        return apiKeys.containsKey(apiKey);
    }
}

// Usage
ApiKeyAuthenticator auth = new ApiKeyAuthenticator();
auth.registerApiKey("user123", "sk_test_1234567890");

if (auth.authenticate(providedApiKey)) {
    // Allow access
} else {
    // Return 401 Unauthorized
}
```

Basic Authentication:

```
public class BasicAuthenticator {
    private final Map<String, String> users = new ConcurrentHashMap<>();

    public String createAuthHeader(String username, String password) {
        String auth = username + ":" + password;
        String encodedAuth = Base64.getEncoder().encodeToString(auth.getBytes());
        return "Basic " + encodedAuth;
    }

    public boolean authenticate(String authHeader) {
        if (authHeader == null || !authHeader.startsWith("Basic ")) {
            return false;
        }

        String base64Credentials = authHeader.substring("Basic ".length());
        String credentials = new String(Base64.getDecoder().decode(base64Credentials));
        String[] parts = credentials.split(":", 2);

        String username = parts[0];
        String password = parts[1];

        String storedPassword = users.get(username);
        return storedPassword != null && storedPassword.equals(password);
    }
}
```

Secure Error Handling:

```
public class SecureErrorHandler {
    public String handleError(Exception e) {
        // Log full error internally
        logger.error("Internal error", e);

        // Return safe message to client
        if (e instanceof ValidationException) {
            return "{\"error\": \"Invalid input provided\", \"code\": \"VALIDATION_ERROR\"}";
        } else if (e instanceof AuthenticationException) {
            return "{\"error\": \"Authentication failed\", \"code\": \"AUTH_ERROR\"}";
        } else {
            // Don't expose internal error details
            return "{\"error\": \"An error occurred\", \"code\": \"INTERNAL_ERROR\"}";
        }
    }
}
```

Programming Principles:

- **Defense in Depth**: Multiple security layers
- **Fail Securely**: Errors don't expose sensitive data
- **Least Privilege**: Minimal access by default
- **Input Validation**: Never trust user input
- **Rate Limiting**: Prevent resource exhaustion

How to Run:

```
mvn exec:java -Dexec.mainClass="com.example.basic.week10ApisSecurity.ApiSecurityDemo"
```

Key Concepts

1. RESTful APIs

REST = Representational State Transfer

Principles:

- **Stateless**: Each request contains all needed information

- **Resource-based**: URLs represent resources, not actions
- **HTTP methods**: Use appropriate verbs for operations
- **Standard formats**: JSON for data exchange

HTTP Methods:

- **GET**: Retrieve data (read-only, safe, idempotent)
- **POST**: Create new resource (not idempotent)
- **PUT**: Update existing resource (idempotent)
- **DELETE**: Remove resource (idempotent)

URL Structure Examples:

```
GET    /api/users          → Get all users
GET    /api/users/123      → Get specific user
POST   /api/users          → Create new user
PUT    /api/users/123      → Update user
DELETE /api/users/123      → Delete user
```

HTTP Status Codes:

- **2xx Success**: 200 OK, 201 Created, 204 No Content
- **4xx Client Error**: 400 Bad Request, 401 Unauthorized, 404 Not Found
- **5xx Server Error**: 500 Internal Server Error, 503 Service Unavailable

2. JSON Format

JSON = JavaScript Object Notation

Data Types:

- **String**: `"hello"`
- **Number**: `42` , `3.14`
- **Boolean**: `true` , `false`
- **Null**: `null`
- **Object**: `{"key": "value"}`
- **Array**: `["item1", "item2"]`

Structure Example:

```
{
    "name": "Alice",
    "age": 28,
    "email": "alice@example.com",
    "address": {
        "city": "New York",
        "zipCode": "10001"
    },
    "skills": ["Java", "Python", "SQL"],
    "isActive": true
}
```

Best Practices:

- Use camelCase for property names
- Keep structure flat when possible
- Validate JSON before parsing
- Handle parsing errors gracefully

3. Security Principles

CIA Triad:

- **Confidentiality**: Protect data from unauthorized access
- **Integrity**: Ensure data hasn't been tampered with
- **Availability**: Keep services accessible to legitimate users

OWASP Top 10 API Security Risks:

1. Broken Object Level Authorization
2. Broken Authentication
3. Broken Object Property Level Authorization
4. Unrestricted Resource Consumption
5. Broken Function Level Authorization
6. Unrestricted Access to Sensitive Business Flows
7. Server Side Request Forgery
8. Security Misconfiguration
9. Improper Inventory Management
10. Unsafe Consumption of APIs

Security Best Practices:

- Always use HTTPS for production
- Validate and sanitize all inputs
- Implement rate limiting
- Use strong authentication
- Log security events
- Keep dependencies updated
- Follow principle of least privilege
- Don't expose sensitive information in errors

Common Mistakes to Avoid

1. Not Handling Exceptions

```
// BAD - no error handling
HttpResponse<String> response = client.send(request,
    HttpResponse.BodyHandlers.ofString());
String body = response.body();

// GOOD - proper error handling
try {
    HttpResponse<String> response = client.send(request,
        HttpResponse.BodyHandlers.ofString());

    if (response.statusCode() == 200) {
        String body = response.body();
        // Process response
    } else {
        System.err.println("HTTP error: " + response.statusCode());
    }
} catch (IOException e) {
    System.err.println("Network error: " + e.getMessage());
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    System.err.println("Request interrupted");
}
```

2. Not Validating Input

```
// BAD - no validation
public void createUser(String email, int age) {
    // Directly use unvalidated input
}

// GOOD - validate before using
public void createUser(String email, int age) throws ValidationException {
    if (email == null || !isValidEmail(email)) {
        throw new ValidationException("Invalid email");
    }
    if (age < 0 || age > 150) {
        throw new ValidationException("Invalid age");
    }
    // Now safe to use
}
```

3. Exposing Sensitive Information

```
// BAD - exposes internal details
catch (SQLException e) {
    return "Database error: Connection to mysql://localhost:3306 failed for user 'admin'";
}

// GOOD - generic error message
catch (SQLException e) {
    logger.error("Database error", e);  // Log internally
    return "An error occurred. Please try again later.";  // Public message
}
```

4. Not Using HTTPS

```
// BAD - HTTP in production
String apiUrl = "http://api.example.com/users";

// GOOD - HTTPS for security
String apiUrl = "https://api.example.com/users";
```

5. Hardcoding Secrets

```
// BAD - hardcoded API key
String apiKey = "sk_live_1234567890abcdef";

// GOOD - use environment variables
String apiKey = System.getenv("API_KEY");
if (apiKey == null) {
    throw new IllegalStateException("API_KEY not configured");
}
```

Practice Exercises

Exercise 1: Weather API Client

Create a simple weather API client that:

- Makes GET requests to a public weather API
- Parses JSON responses
- Handles errors gracefully
- Implements basic caching to reduce API calls

Exercise 2: User Management API

Build a user management system with:

- Create, read, update, delete operations
- Input validation for all fields
- Rate limiting (5 requests per minute per user)
- Basic authentication
- Secure error handling

Exercise 3: API Security Audit

Review existing code and:

- Identify security vulnerabilities
- Implement input validation

- Add authentication
- Implement rate limiting
- Secure error messages

Security Checklist

Use this checklist when building APIs:

Input Validation:

- [ ] Validate all user inputs
- [ ] Sanitize data before processing
- [ ] Check field lengths and types
- [ ] Validate email formats, URLs, etc.

Authentication & Authorization:

- [ ] Implement authentication mechanism
- [ ] Verify permissions for each request
- [ ] Use secure password storage (hashing)
- [ ] Implement session management
- [ ] Support token expiration

Rate Limiting:

- [ ] Implement rate limiting per user/IP
- [ ] Return 429 status for exceeded limits
- [ ] Provide retry-after headers
- [ ] Monitor for abuse patterns

Error Handling:

- [ ] Catch all exceptions
- [ ] Log errors internally
- [ ] Return generic error messages
- [ ] Don't expose stack traces
- [ ] Use appropriate HTTP status codes

Data Protection:

- [ ] Use HTTPS for all communications
- [ ] Encrypt sensitive data at rest
- [ ] Don't log sensitive information
- [ ] Implement proper access controls

Monitoring & Logging:

- [ ] Log all security events
- [ ] Monitor for suspicious activity
- [ ] Set up alerts for anomalies
- [ ] Regular security audits

Additional Resources

- **Java Documentation**: HttpClient Guide
- **org.json Library**: JSON in Java
- **REST API Design**: Best practices and conventions
- **OWASP**: API Security Top 10

Remember: Security is not a feature you add later—it must be built in from the start. Always validate input, implement proper authentication, and handle errors securely!